CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

Chandubhai S Patel Institute of Technology

DEPSTAR

ADVANCED WEB DEVELOPMENT FRAMEWORKS (ITUE301)

Practical 3: API Integration and Data Rendering in React

CO/PO	CO1 / PO3, PO5
Objective	To consume a REST API in React and handle asynchronous data with loading and error states.
Prerequisites	• Practical 1 and 2 completed multi-route portfolio app with state management • Basic understanding of JavaScript Promises and async/await
Theory Concepts (Self-Study Reference)	
Coursera Pointer	See Week 3 IBM: Developing Front-End Apps with React, Module 3 (React hooks, form management, connecting to external servers, async data fetching).
Architecture / Diagram	

Diagram	`Projects.jsx`
	`├── useEffect()  → triggers fetch on mount`
	`├── useState: data, loading, error`
	`├── [loading] → <Spinner />`
	`├── [error]   → <ErrorMessage />`
	`└── [success] → <RepoList data={data} />`

Problem Definition	
Problem Statement	• Integrate a public REST API (e.g., GitHub API) into the portfolio application to fetch and display a list of repositories dynamically. • Use the Projects page built in Practical 2 as the integration point. • Implement a loading spinner component shown while the request is in progress. • Implement an error message component shown if the API call fails. • Render at least the repository name and URL for each item returned.
Key Questions / Analysis	• Why is useEffect required to trigger a fetch on component mount instead of calling fetch directly in the component body? • What is the difference between a loading state and an error state, and why must both be handled separately? • How would the user experience change if loading and error states were not implemented?
Supplementary Problems	• Add a retry button that re-triggers the fetch when an error occurs. • Add a search input that filters the rendered repository list by name. • Display the repository's star count alongside its name.
Key Skills Addressed	• Fetching data from a REST API using fetch or axios • Using useEffect to trigger side effects on component mount • Managing asynchronous state loading, success, and error • Conditionally rendering UI based on request state
Learning Outcome	Students will be able to consume a REST API in React, manage asynchronous state correctly, and provide appropriate UI feedback during all stages of a network request.
Lab Session Step-by-Step	
Steps	1. Inside Projects.jsx, set up 3 state variables for data, loading, and error:

```
const [repos, setRepos] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);
```

2. Use useEffect to fetch data when the component mounts:

```
useEffect(() => {
  fetch('https://api.github.com/users/<username>/repos')
    .then((res) => res.json())
    .then((data) => setRepos(data))
    .catch((err) => setError(err.message))
    .finally(() => setLoading(false));
}, []);
```

3. Conditionally render based on state:

```
if (loading) return <Spinner />;
if (error) return <ErrorMessage message={error} />;
```

4. Map over the repos array and render name and html_url for each repository.
5. Test the happy path confirm data renders correctly after loading.
6. Test the error path temporarily break the API URL and confirm the error message displays.

Tools / Technology / Resources Required	• Node.js (v18+), npm • Visual Studio Code • React 18+, Fetch API or Axios • Browser DevTools (Network tab)
Total Hours	• Lab Session: 2 hours • Implementation: 1.5 hours • Testing / Modification: 0.5 hours • Total Engagement: 2 hours

Faculty Teaching Guide

Guidance Notes	• Use a public API requiring no authentication (GitHub API is ideal) to avoid API key issues in lab. • Demonstrate breaking the API URL deliberately to show the error state students rarely test this on their own. • Explain why the dependency array [] in useEffect matters fetching on every render is a very common bug to anticipate. • Walk through the difference between .then() chains and async/await if students choose to use the latter.
Common Mistakes	• Calling fetch directly inside the component body without useEffect, causing infinite re-fetching. • Omitting the dependency array in useEffect, causing the request to fire on every render. • Not handling the error case at all, causing the app to crash or show a blank screen on failure. • Forgetting to set loading back to false after the fetch completes, leaving the spinner stuck.

Troubleshooting Guide

Symptom	Likely Cause	Fix
Infinite network requests in console	Missing dependency array in useEffect	Add an empty array [] as the second argument to useEffect
Spinner never disappears	setLoading(false) not called in .finally() or after both success/error paths	Use .finally() to guarantee loading is set to false regardless of outcome
App crashes on API failure	No error boundary or conditional check before rendering data	Add error state handling and conditionally render ErrorMessage before the data list

CORS error in console	API does not allow cross-origin requests from localhost	Use a CORS-friendly public API like GitHub's; do not attempt to bypass CORS with browser flags

Student Assignment / Post Laboratory Work

Tasks	• Review the Theory Concepts section and the assigned Coursera module before next week. • Add a retry button that re-triggers the fetch when an error occurs. • Add a simple search/filter input above the repository list.

GitHub Deliverables

Expected Repository Contents	• Same repository as Practical 1 and 2, updated with new commits • Working API integration on the Projects page with loading and error states • At least one commit specifically for API integration • Updated README noting the API used and any setup steps

Rubrics (Practical Evaluation / Viva)

Criteria (Marks)	Description
Conceptual understanding (2 Marks)	Explains what and why, not just what was done
Implementation (4 Marks)	Code written, runs, and is original
Correctness (2 Marks)	Output matches expected result
Reflection (1 marks)	Errors documented and explained
Lab file evidence (1 marks)	Completeness and presentation
Total: 10 marks	**Passing Threshold: 5 / 10 (50%)**